Simulation Modelling Practice and Theory 68 (2016) 146–162

Contents lists available at ScienceDirect

Simulation Modelling Practice and Theory

journal homepage: www.elsevier.com/locate/simpat

Top-Down Characterization Approximation based on performance counters architecture for AMD processors

Mateusz Jarus[a,*], Ariel Oleksiak[a,b]

[a] Poznan Supercomputing and Networking Center, Energy Efficient ICT Laboratory, ul. Noskowskiego 10, 61-704 Poznań, Poland
[b] Poznan University of Technology, Department of Computer Science, Pl. Marii Skodowskiej-Curie 5, 60-965 Poznań, Poland

ARTICLE INFO

Article history:
Received 22 April 2016
Revised 26 August 2016
Accepted 27 August 2016
Available online 7 September 2016

Keywords:
Processor performance analysis
Applications
Modeling
Energy efficiency

ABSTRACT

Due to the increasing complexity of the processors, developers often seek for tools that would simplify the process of finding bottlenecks while executing applications. Although more and more data may be collected from processors, usually much detailed knowledge about the internals of a given architecture is required to understand them.

This paper introduces a Top-Down Characterization Approximation for the analysis of applications performance executed on AMD processors and is an extension of a Top-Down Method initially developed by Intel. Since not all required performance counters are available on AMD processors to calculate the exact values of metrics, this method was named as an approximation. It allows one to get a deeper understanding of different stages of program execution, compare different architectures and identify bottlenecks in out-of-order processors. It hides from the user the complexity of microarchitecture details and at the same time exposes the main contributors of inefficient program execution. This method aims at defining a few main metrics on top of performance counters to easily locate the main efficiency issues.

At this time this method was applied to Intel processors only. The main reason behind it was the fact that it uses designated performance counters that are unique among different processors and its portability is not straightforward. Positive feedback from users encouraged the authors to develop a similar technique for AMD processors.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

Presented method aims at easily finding true bottlenecks of applications in out-of-order AMD processors. Described metrics form a tree hierarchy in which each node represents certain performance issues at a given level of granularity. The common denominator of all of the metrics is the number of cycles the processor spent on performing specific actions.

Although the inspiration for this work came from the method developed by Intel [1], not all of the metrics could be transferred to AMD Top-Down Characterization. This is due to the fact that AMD processors have a different set of available performance counters. The four uppermost metrics have been preserved but the next levels of the hierarchy have been adapted to the data available on AMD processors. The originally proposed submetrics are not the only correct ones. Different metrics may be proposed depending on the architecture and the data available on a given processor. For example, Branch

* Corresponding author.
E-mail addresses: jarus@man.poznan.pl (M. Jarus), ariel@man.poznan.pl (A. Oleksiak).

http://dx.doi.org/10.1016/j.simpat.2016.08.006
1569-190X/© 2016 Elsevier B.V. All rights reserved.

Resteers metric counts the cycles the CPU was stalled due to switches from DSB to MITE pipelines, which is characteristic for Intel processors only (introduced with the Sandy Bridge microarchitecture). Therefore, this paper aims at developing an analogous method to Intel's Top-Down Approach with the use of performance counters currently available on AMD processors, while maintaining the same principles. Since not all performance counters are available to calculate the exact values of all metrics, some fixed costs had to be introduced and thus this method was named as an approximation. However, fixed costs were also introduced in the original Intel Top-Down Method, e.g. to calculate L3 latency.

Presented method may serve program developers to easily find bottlenecks in the processor. A clear definition of a structure of metrics gives information about the performance of executed programs, without the need to have a depth understanding of the internals of microprocessors or without a detailed analysis of all of the available performance counters. It also allows to compare the efficiency of several processors executing the same benchmark and at the same time gives information about the places of inefficiencies. Instead of one value that characterizes the benchmark, such as MIPS or FLOPS, presented method may give an answer to the question where the inefficiencies inside the processor arise.

Moreover, the presented method may be an incentive for AMD to further enrich their future microprocessors with more performance counters that may extended the list of metrics. This would allow to get an even deeper insight into the execution of programs. The popularity of the original Intel Top-Down Method suggests that it was warmly welcomed by developers of applications and testers of hardware. This experience may be used by AMD company to further develop the method proposed in this paper. Although Intel dominated both the server and desktop markets, AMD company still offers very competitive solutions, with its almost $4 billion revenue in 2015[1]. The upcoming Zen architecture will have its premiere in 2016 in the desktop, laptop and server markets. As the officials suggest, the upcoming products will be very competitive and may significantly improve the position of AMD on the market. Offering their clients a method that easily allows to identify bottlenecks in their applications may be an additional encouragement to choose AMD products.

The greatest value of this paper is a clear definition of the metrics in the AMD Top-Down Characterization based on the existing performance counters in the AMD 15h, Opteron or R-Series Family processors. To our knowledge, it is the first attempt to transfer Intel Top-Down Method to AMD processors. The AMD CodeAnalyst tool, according to its website [2], offers Top-Down performance analysis. What they mean by this is not, however, the access to metrics that allow to easily find the bottlenecks in applications. CodeAnalyst allows to check the functions of the code that generate the highest values of performance counters. Then it is possible to drill down to selected function and monitor the values of performance counters for it. It is therefore a graphical tool mainly for analyzing raw performance counters. In case of the naïve-approaches the number of some events (such as cache accesses) are only taken into account with fixed costs for them. Therefore, they do not count the exact cycles when the resources are idle but try to estimate these times.

We have demonstrated our method on AMD R-Series and AMD Opteron Family processors.

The rest of this paper is organized as follows. Section 2 provides a background and presents the main assumptions of Top-Down Characterization. Section 3 presents the details of AMD Top-Down Characterization method and the definitions of metrics. Results on popular workloads and different microarchitectures are presented in Section 4. Related work is discussed in Section 5. Finally, Section 6 concludes the paper and outlines future work.

2. Background

The idea behind this method is the same as in the original Intel's work. In order to maintain consistency of the paper the main assumptions are repeated.

Current microprocessors are equipped with mechanisms that improve the efficiency of execution but at the same make the analysis of applications' execution more and more complex. The methodology presented in this paper utilizes the fact that modern out-of-order CPUs are divided into two portions: a Frontend and a Backend. The Frontend fetches instructions from memory, decodes them into micro-operations and sends them to the Backend. The Backend is responsible for executing and retiring them.

Traditional (naïve) approaches to CPU performance monitoring are based on estimations of stalls in cycles. However, due to a few effects they are usually not precise. First of all, stalls overlap, making it possible for some of the units of the processor to work concurrently. Naïve-approaches do not take these effects into account. Secondly, out-of-order processors may execute and retire multiple micro-operations in a single cycle. Finally, penalties are workload-dependent, therefore fixed penalties for all workloads are not precise. Due to the lack of some performance counters, even in the original Intel Top-Down Method some metrics were calculated using fixed costs. For this reason it was named as an approximation. These included L3 Cache Latency (up to Haswell architecture) or DTLB load and store metrics. However, this method does not heavily rely on fixed costs, they are minimized and used only when metrics cannot be defined in other way.

These issues were addressed by the Top-Down Method by focusing on the program execution at the micro-operation level (taking into account the available pipeline slots) and considering for the calculation of metrics only processor's subdurations during which specific parts of the processor were fully underutilized.

[1] http://ir.amd.com/phoenix.zhtml?c=74093&p=irol-newsArticle&ID=2130467

[2] http://developer.amd.com/tools-and-sdks/archive/compute/amd-codeanalyst-performance-analyser/amd-codeanalyst-performance-analyser-for-linux/

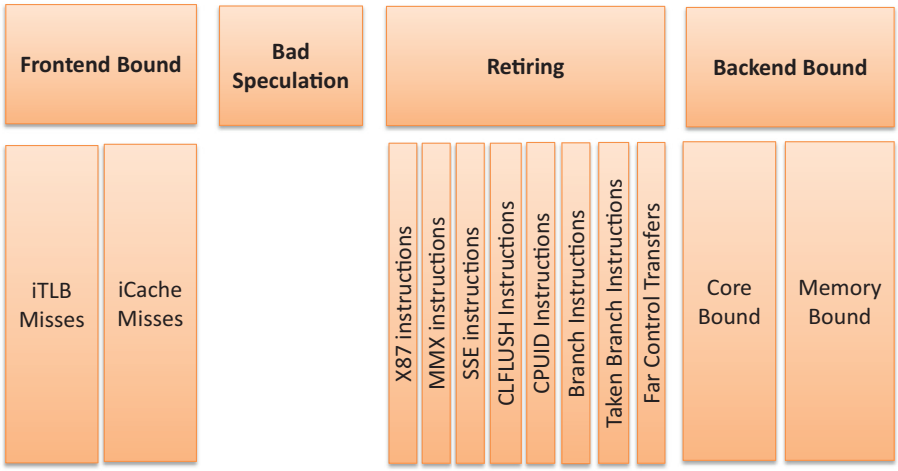


Fig. 1. AMD Top-Down Characterization.

3. Proposed AMD Top-Down Hierarchy

The Top-Down Method is composed of metrics that form a tree structure. Each level of the tree corresponds to a specific level of generality of the analysis. Each metric determines the fraction of time a given unit in a processor was either used or underutilized during program execution. The higher the value of a given metric, the more attention should be given to it by a developer in order to optimize the execution.

The AMD Top-Down Hierarchy is presented in Fig. 1. The uppermost level is composed of four categories: Frontend Bound, Backend Bound, Bad Speculation and Retiring. Every instruction executed on the processor gets classified into one of them. As the Frontend and Backend naturally separate the processor, they were selected as the first two metrics in the hierarchy. If the pipeline-slot does not reach this point of the processor, it means it was bounded by the Frontend. Otherwise, it may get retired or canceled. In such case it will be attributed to Retiring or Bad Speculation category, respectively. In any other case it will attributed to Backend Bound category. An example of such scenario is when the processor waits for data to be fetched from memory.

As the processors may issue multiple micro-operations per cycle, this classification is performed at the granularity of pipeline slots, making it very accurate.

3.1. Frontend Bound category

Frontend represents the first part of the processor where the instructions are fetched from the memory, the next address to fetch is predicted by the branch predictor and instructions are decoded into micro-operations, executed later by the Backend. When no instructions are passed from Frontend to Backend, such a cycle is marked as Frontend Bound. Ideally, the Frontend may issue 4 micro-operations every cycle. Examples of such unutilized slots are stalls due to instruction cache misses.

In the original Intel's work this metric was represented as the value of performance counter called *IDQ_UOPS_NOT_DELIVERED.CORE*. It counts the number of undelivered micro-operations from the Frontend to the Resource Allocation Table while the Backend is not stalled. The most accurate equivalent in case of AMD processor for the *Frontend Bound* category is the *DECODER_EMPTY* performance counter. It counts the number of cycles where the decoder has nothing to dispatch. This is the unit that connects the Frontend and Backend. When it is empty, it means that the execution of the program is bounded by the Frontend – these may be cycles waiting on an instruction fetch that missed the instruction cache.

In case of the original Intel Top-Down Approach if both the Frontend and the Backend are stalled, then the slot will be classified as Back-End Bound. This is because, in that case, fixing the stall in the Frontend would most likely not help an applications performance. The Backend is the bottleneck and it would need to be removed first before fixing issues in the Frontend would have any effect. In case of the proposed in this paper AMD method it is necessary to note that the *DECODER_EMPTY* does not take into account whether the Backend is stalled or not. Therefore, if Frontend Bound is understood in the same manner as the one in the original Intel Top-Down Approach, it may introduce some inaccuracy. In case of Intel processors, the *IDQ_UOPS_NOT_DELIVERED* performance counter, used to calculate the Frontend Bound category, internally counts only the number of undelivered *UOPS* from the Frontend to the Resource Allocation Table when the Backend of the machine is not stalled. Therefore, no other mathematical operations are required. In case of AMD processors unfortunately no other performance counter is available that could be used to fix these differences. No performance counter is either available to calculate the difference between the expected and received value.

The Frontend Bound category is further divided into *Instruction Cache Misses* and *Instruction Translation Lookaside Buffer (ITLB) Misses*. The first of these two counts the number of requests that miss in the instruction cache. These may be defined with the use of *INSTRUCTION_FETCH_STALL* performance counter. It effectively counts the number of cycles the instruction fetcher is stalled for the core.

The second subcategory of the Frontend counts the number of misses in the Instruction Translation Lookaside Buffer. If the requested address is not in the TLB, it is a miss and the translation proceeds by looking up the page table. It is called a page walk. Unfortunately the AMD processors count only the number of ITLB misses, not the number of lost cycles. Similarly to the original Intel Top-Down Approach, in the case of this metric a fixed cost of a miss was defined at the value of 7 cycles. This cost is later multiplied by the number of two performance counters *L1_ITLB_MISS_AND_L2_ITLB_MISS*.

Due to the missing performance counters, the Frontend Bound category is not divided into any other metrics, as in the original Intel Top-Down Method, where some additional subcategories may be found.

3.2. Bad Speculation category

*Bad Speculation*counts the number of slots lost due to incorrect speculations. These are mainly accounted for by the mispredicted branch instructions. Originally this metric was defined as the number of all microoperations issued minus those that eventually retired. The rest of them were categorized under Bad Speculation category. The AMD processors do not have any performance counter that counts the issued microoperations. Therefore, in this paper an approach of counting all of the events related to branch misprediction is presented. As AMD processors give only information about the number of events, not cycles used by them, also in this metric the cost of these events is introduced. This is the second metric, next to ITLB misses, and the last one in this AMD Top-Down Characterization that uses the additional cost value. The specific value of this cost depends on the given processor. In the experiments presented in this paper a value of 12 cycles was used, according to practical experiments [2]. These two metrics, however, have relatively low absolute values, therefore the inaccuracy introduced by fixed costs is relatively low.

The Intel Top-Down Approach captures not only the cycles wasted after the execution of the branch misprediction, i.e. used to flush the pipeline, but also shows the cycles from the issue of the mispredicted branch. The exact formula of Bad Speculation metric on Intel is: $(UOPS_ISSUED.ANY - UOPS_RETIRED.RETIRE_SLOTS) + Pipeline_Width * INT_MISC.RECOVERY_CYCLES$. The first part of the formula $UOPS_ISSUED.ANY - UOPS_RETIRED.RETIRE_SLOTS$ is the largest contributor (more than two thirds) to the value of this metric. For example, in case of the bzip2 application out of the 14.63% of Bad Speculation on Intel i5-4210M 10.62% comes from calculating not retired microoperations, while 4.01% from $Pipeline_Width * INT_MISC.RECOVERY_CYCLES$. Similar results were obtained for the rest of the benchmarks, with the highest proportion of not retired microoperations for hmmer (the whole Bad Speculation metric had a value of 11.45% and 8.91% came from not retired microoperations).

The *Pipeline_Width* is equal to 4 and it means the number of instructions able to be issued, executed or completed per cycle. Thanks to such definition of this metric it is therefore not required to take into account the number of stages in the pipeline (i.e. the length of the pipeline, which in the case of Haswell processor is between 14 and 19).

Because there is no performance counter on AMD processors that calculates the issued microoperations, the proposed formula for AMD takes into account the length of the pipeline. This number approximates the cost of the whole execution of a mispredicted branch, not only cycles used to flush the pipeline. However, it is important to note that this is not an equivalent of $Pipeline_Width * INT_MISC.RECOVERY_CYCLES$ from Intel Top-Down metric. It takes into account the whole length of the pipeline.

3.3. Retiring category

This category represents microoperations that eventually get retired. The value of 100% means that all of the microoperations pass completely through the pipeline and get retired during execution of a benchmark. The higher value is reached in this category, the more efficient are the computations. However, a high Retiring value does not necessarily mean that there is no space for optimization. Reducing the number of instructions that need to be executed in the program is one the techniques that may also bring benefits.

The *Retiring* category is represented with the use of *RETIRED UOPS* performance counter, similarly to the Intel Top-Down Method, where the *UOPS_RETIRED.RETIRE_SLOTS* is used.

The Retiring metric may be further divided into 8 other submetrics. These define specific microoperations that get retired during execution and are the following:

1. *x87 instructions* – the number of x87 floating point microoperations that have retired
2. *MMX instructions* – the number of MMX instructions retired (MMX is a single instruction, multiple data (SIMD) instruction set)
3. *SSE instructions* – the number of SSE instructions retired (SSE, SSE2, SSE3, SSSE3, SSE4A, SSE4.1, SSE4.2, AVX, XOP, FMA4)
4. *CLFLUSH* – the number of CLFLUSH instructions retired
5. *CPUID* – the number of CPUID instructions retired
6. *Branch instructions* – the number of branch instructions retired. This includes all types of architectural control flow changes, including exceptions and interrupts

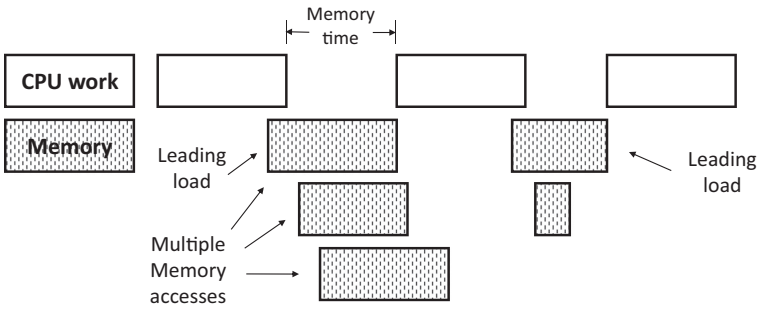


Fig. 2. An example of leading loads.

7. *Taken branch instructions* – the number of taken branches that were retired. This includes all types of architectural control flow changes, including exceptions and interrupts
8. *Far control transfers* – the number of far control transfers retired including far call/jump/return, IRET, SYSCALL and SYSRET, plus exceptions and interrupts

Specific performance counters used to define these metrics are presented in Section 3.5.

3.4. Backend Bound category

Backend represents the second part of the processor where the instructions are allocated from the Frontend and which is responsible for monitoring when microoperation data operands are available and executing the microoperation in an available execution unit. Backend Bound reflects the situations when no microoperations are delivered at the issue pipeline as the Backend runs out of some resources, e.g. load buffers.

The *Backend Bound* category is calculated as 100% minus the sum of all three other metrics the top of the hierarchy. Since a microoperation may get classified only into one of the four top categories (Frontend Bound, Bad Speculation, Retiring or Backend Bound), calculating Backend Bound in this way gives correct results and is the simplest method.

Backend Bound is further split into *Memory Bound* and *Core Bound*. These submetrics are determined using events corresponding to the utilization of the execution units. Stalls in the Memory Bound category have causes related to the memory subsystem. These are accounted for by cache misses and memory accesses. Core Bound stalls are caused by a non-optimal use of the available execution units in the CPU in each cycle. For example, some multi-cycle divide instructions competing for the divide units could cause Core Bound stalls. If the memory requirements are not significant, similar scenarios may occur. For example, a loop doing Floating Point (FP) arithmetic calculations on data that fits within cache.

The Memory Bound category is easily calculated on Intel processors with the use of *CYCLE_ACTIVITY.STALLS_LDM_PENDING* performance counter. This event counts cycles during which no instructions were executed in the execution stage of the pipeline and there were memory instructions waiting for data. In case of the AMD processors this performance counter is not available. However, it is possible to estimate the memory accesses that caused stalls to the core. It is not sufficient to count all the memory accesses and multiply them by some predefined cost because processors can exploit memory-level parallelism (MLP) by overlapping multiple cache misses. Finally, DRAM access latency varies with access patterns, making it difficult to predict time spent waiting on memory from access counts alone.

However, it is possible to estimate the memory accesses that stalled the core with the use of the leading loads model. It was simultaneously theoretically defined by three groups [3–5]. In their approach they utilized the observation that although many memory accesses may be outstanding, only one can stall the pipeline. Therefore, the first nonspeculative load that misses in the last level of the cores cache is considered a *leading load*. The memory time is calculated as the time between the miss and when the data is returned to the processor. All other misses until this load returns are not leading loads and are ignored. Fig. 2 presents this scenario.

Although the assumption that the other memory accesses will not eventually stall the pipeline is a simplifying assumption, it was practically validated and proves to give accurate results. On AMD processors a Miss Address Buffer (MAB) was introduced [6], which is a structure that tracks the outstanding cache misses. With each cache miss one of the entries will be used to hold the memory request. It is cleared when the data returns. These entries are allocated with a fixed priority. The highest priority entry that does not currently hold a request is always used to keep the newest memory request. Therefore, the entry with the highest priority holds information about the leading memory requests. Moreover, the time each entry (and thus, also the leading load entry) is occupied in cycles may be measured directly using a specified hardware performance counter MAB_WAIT.

In Family 15h cores the MAB occupancy is measured for L2 cache misses. Therefore, if there is no L3 cache, using the leading loads model the Memory Bound metric gives effectively information only about the external memory boundedness. In order to take into account also the cache access time, additional information needs to be collected.

AMD introduced a new profiling technique that provides precise profile information – Instruction Based Sampling (IBS) [7]. The traditional performance counter sampling, called Event Based Sampling (EBS), gives information about the number

Table 1
Fixed costs in cycles for selected metrics on AMD processor and definition of auxiliary slots value used in the definitions of metrics.

Cost	Value
ITLB miss cost	7
Branch misprediction cost	12
Slots	UNHALTED_REFERENCE_CYCLES * 4

of pre-configured events. The number of events that can be measured in a single performance test is limited by the number of counters. Moreover, it is difficult to relate a hardware event to the instruction that triggered it because the restart address is not the location after the trigger instruction. IBS, on the other hand, selects an operation periodically based on a configurable sampling period. The selected operation and all events caused by the operation are monitored and recorded as it proceeds through the pipeline. This mechanism allows to, among many others, retrieve the information about the number of cycles spent on a cache miss.

The information about the number of cycles used to retrieve the data from the cache memory may not be accurate after a single program run. IBS selects periodically (according to a preconfigured sampling period) operations to monitor. To get a more accurate value, several executions should be performed to calculate an average time of memory access for a given benchmark. This average value multiplied by the number of data cache misses (retrieved from performance counter *DATA_CACHE_MISSES*) gives the whole time in cycles spent to retrieve data from cache to processor. It does not take into account the effect of memory-level parallelism. As a result, not all monitored cache accesses stall the processor. Therefore, the received value would inform whether the benchmark intensively references cache memory, not whether it is cache bound.

It should not be therefore included in the presented Top-Down Characterization as it does not hold the property of real memory boundedness. It may, however, serve as a value that facilitates the analysis of program execution, next to the Memory Bound metric. An analyzed benchmark may have a low value of Memory Bound metric but at the same time may indicate high number of cache accesses. Although it is not the same as cache boundedness, it may be and should definitely bring the attention of the developer to the issue of memory usage of the analyzed benchmark. Together with the Memory Bound metric it gives information whether the program heavily uses only external memory, both cache and external memory or only cache.

If the benchmark is not memory bounded, it is assumed to be core bounded, similarly to the approach used in the original Intel Top-Down Method. Therefore, the Core Bound category is calculated as Backend Bound minus Memory Bound.

3.5. Metrics definitions

This section defines the specific performance counters used to calculate the metrics in the AMD Top-Down Characterization.

Table 1 presents fixed costs for the ITLB misses and branch mispredictions on AMD processors. These values were selected based on practical experiments performed by Fog [2]. This table presents also the definition of auxiliary slots value which is used in the definition of metrics and which represents the available pipeline slots in each processor cycle.

Table 2 presents the definitions of all AMD Top-Down Characterization metrics based on the performance counters available in AMD 15h Family processors. The choice of these specific performance counters was already described in previous sections.

4. Results

In this section we present Top-Down Characterization results for a few scenarios in which the practical possibilities of using presented method are suggested. These sections:

- give an overview of the values of metrics for different integer and floating point benchmarks from the SPEC CPU2006 suite (Section 4.1),
- present the possibilities for monitoring different stages of program execution (Section 4.2),
- show the difference in the values of metrics for optimized and non-optimized versions of code (Section 4.3),
- present the analysis of running benchmarks on multiple cores and multiple benchmarks on one core (Section 4.4),
- compare different architectures with the use of presented method (Section 4.5),
- show correlation between the values of metrics and energy consumption (Section 4.6).

The system parameters are presented in Table 3.

The Linux perf tool was used to collect the values of performance counters. The event names to event numbers were mapped using the libpfm4 library. It is a helper library to help encode performance events to use with the operating system kernels performance monitoring interfaces. In the experiments all of them were measured system-wide. Linux kernel 3.10 was used for the experiments.

Table 2
Definitions of AMD Top-Down Characterization metrics.

Metric	Definition
Top-level metrics	
Frontend Bound	DECODER_EMPTY/slots
Bad Speculation	(RETIRED_MISPREDICTED_BRANCH_INSTRUCTIONS + RETIRED_TAKEN_BRANCH_INSTRUCTIONS_MISPREDICTED + RETIRED_INDIRECT_BRANCHES_MISPREDICTED) ∗ (Branch misprediction cost)/UNHALTED_REFERENCE_CYCLES
Retiring	RETIRED_UOPS/slots
Backend Bound	100% – (Frontend Bound + Bad Speculation + Retiring)
Frontend Bound subcategories	
ICache Misses	INSTRUCTION_FETCH_STALL/slots
ITLB Misses	(L1_ITLB_MISS_AND_L2_ITLB_HIT + L1_ITLB_MISS_AND_L2_ITLB_MISS) ∗ (iTLB miss cost)/UNHALTED_REFERENCE_CYCLES
Backend Bound subcategories	
Memory Bound	(MAB_WAIT:0/UNHALTED_REFERENCE_CYCLES)
Core Bound	Backend Bound – Memory Bound
Retiring subcategories	
x87 instructions	RETIRED_MMX_FP_INSTRUCTIONS:X87
MMX instructions	RETIRED_MMX_FP_INSTRUCTIONS:MMX
SSE instructions	RETIRED_MMX_FP_INSTRUCTIONS:SSE
CLFLUSH instructions	RETIRED_CLFLUSH_INSTRUCTIONS
CPUID instructions	RETIRED_CPUID_INSTRUCTIONS
Branch instructions	RETIRED_BRANCH_INSTRUCTIONS
Taken branch instructions	RETIRED_TAKEN_BRANCH_INSTRUCTIONS
Far control transfers	RETIRED_FAR_CONTROL_TRANSFERS

Table 3
AMD R system parameters for the tests.

Processor	AMD R-464L, 2.3GHz frequency, 4 cores, 2 x 64KB L1 instruction cache, 4 x 16KB L1 data cache, 2 x 2MB L2 data cache
Memory	16GB DDR3 @ 800MHz
OS	CentOS Linux release 7.1.1503
Benchmark	SPEC 2006CPU v1.2
Compiler	gcc v4.8.3

In a few experiments some of the cores were turned off by writing to the file /sys/devices/system/cpu/online.

To calculate all of the first-level metrics, six performance counters need to be collected simultaneously. As AMD allows to measure only four at the same time [7], multiplexing is used to collect all of the data. The hardware counters are time-shared with a subset of events. Each of them is monitored for a fraction of time, then each event trace is linearly interpolated between unobserved samples. The accuracy of this method depends on different factors, mainly on the regularity of the executed benchmark. If the application has similar repeatable stages, the preciseness increases. In the case of the benchmark analyzed in this paper the difference in values of metrics between 10 executions of the same benchmark were never higher than 2.5%. Therefore, we may assume that one application run is enough to calculate the metrics.

4.1. Results for the SPEC CPU2006 suite

The values of the four top metrics for the SPEC CPU2006 suite are presented in Fig. 3. Integer benchmarks contains the first 11 programs (from bzip2 to xalancbmk), while floating point benchmarks are the rest (from bwaves to sphinx3). Fig. 4 presents the results for Memory Bound and Core Bound metrics.

For the analyzed benchmarks the Retiring metric has a value between 14.13% (mcf benchmark) and 37.97% (h264ref) and correlates with the Instructions Per Cycle ratio. Mcf is a program used for single-depot vehicle scheduling in public mass transportation. The relatively high value of Bad Speculation suggests that the processor often needs to deal with branches during execution of the program. The developer of the application may try to minimize the branches in the code to improve the CPU branch prediction. The Backend Bound metric has a value of 42.74%. Taking a look at the subcategories it turns out that mcf has a large value of Memory Bound metric – 15.82% (second largest value among the analyzed benchmarks). It suggests that memory might be a bottleneck in this case.

4.2. Values of metrics throughout application run

The developer of the application may also use this method to analyze the metrics throughout application run. As the program may have different stages during execution, the values of presented metrics may also change. This analysis may suggest which part of the application underperforms. Fig. 5 presents values of metrics Retiring, Frontend Bound, Bad Speculation and Backend Bound for the leslie3d benchmark. It is easily visible that after around 15 s a different stage of

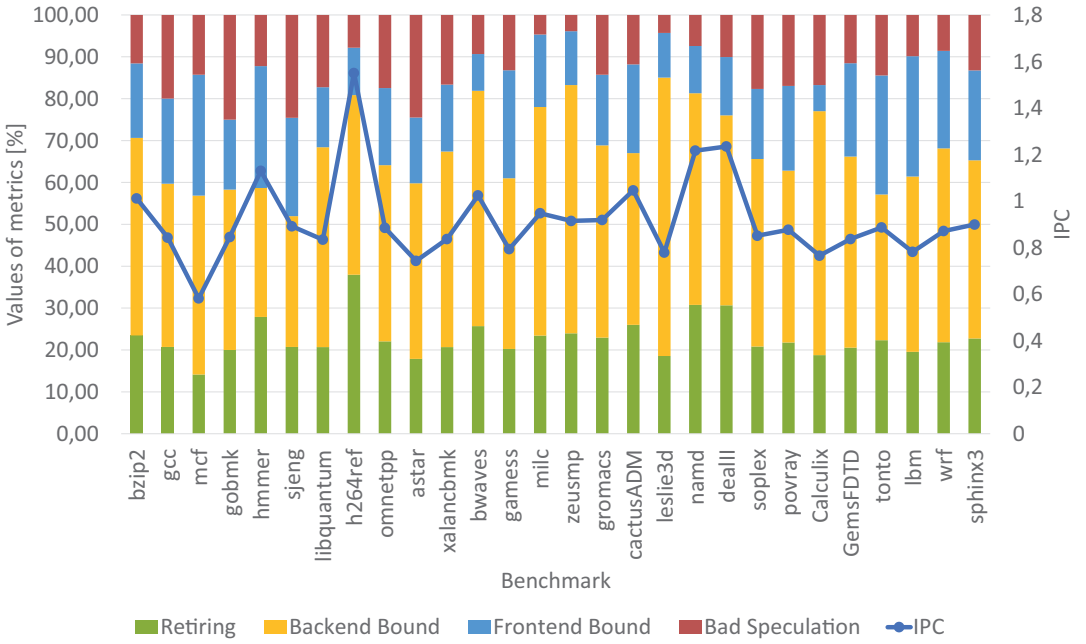


Fig. 3. Retiring, Backend Bound, Frontend Bound, Bad Speculation and IPC metrics for the SPEC CPU2006 suite on AMD R processor.

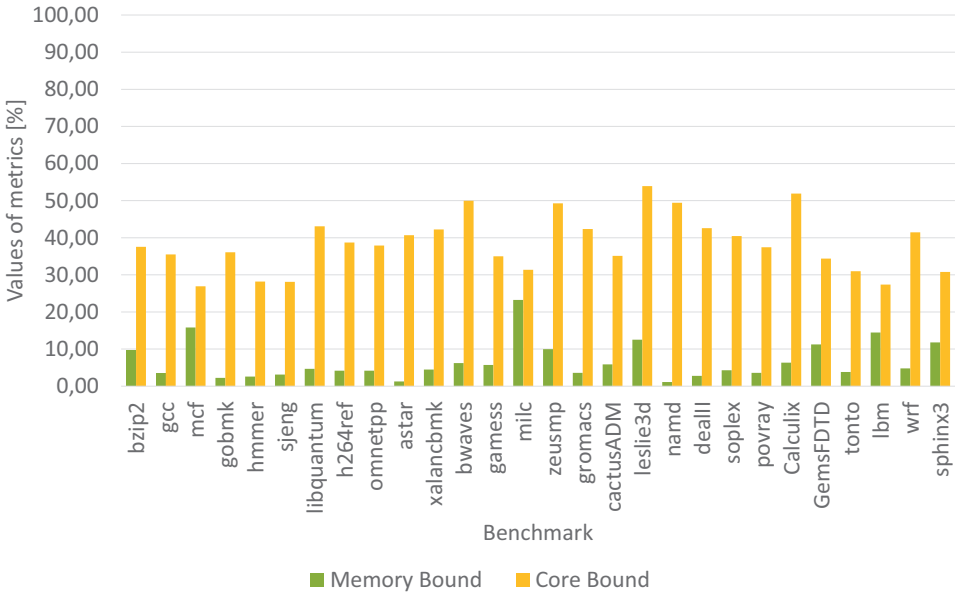


Fig. 4. Memory Bound and Core Bound metrics for the SPEC CPU2006 suite on AMD R processor.

application execution started. The developer of the program may optimize parts of the code that are executed only at that stage.

4.3. Matrix multiplication

The case presented below describes a classic, textbook approach to matrix multiplication. The algorithm has well-known memory access performance issues. The source code of the first implementation is presented by Algorithm 1, later called **for1** in this paper. The memory issues in this case arise from non-sequential accesses to arrays. The elements within a row of an array are arranged sequentially in memory. The fastest changing array index in matrix_*b*, *k* (bolded in the algorithm in line 4), accesses a different row of matrix_*b* in each iteration, causing many cache misses.

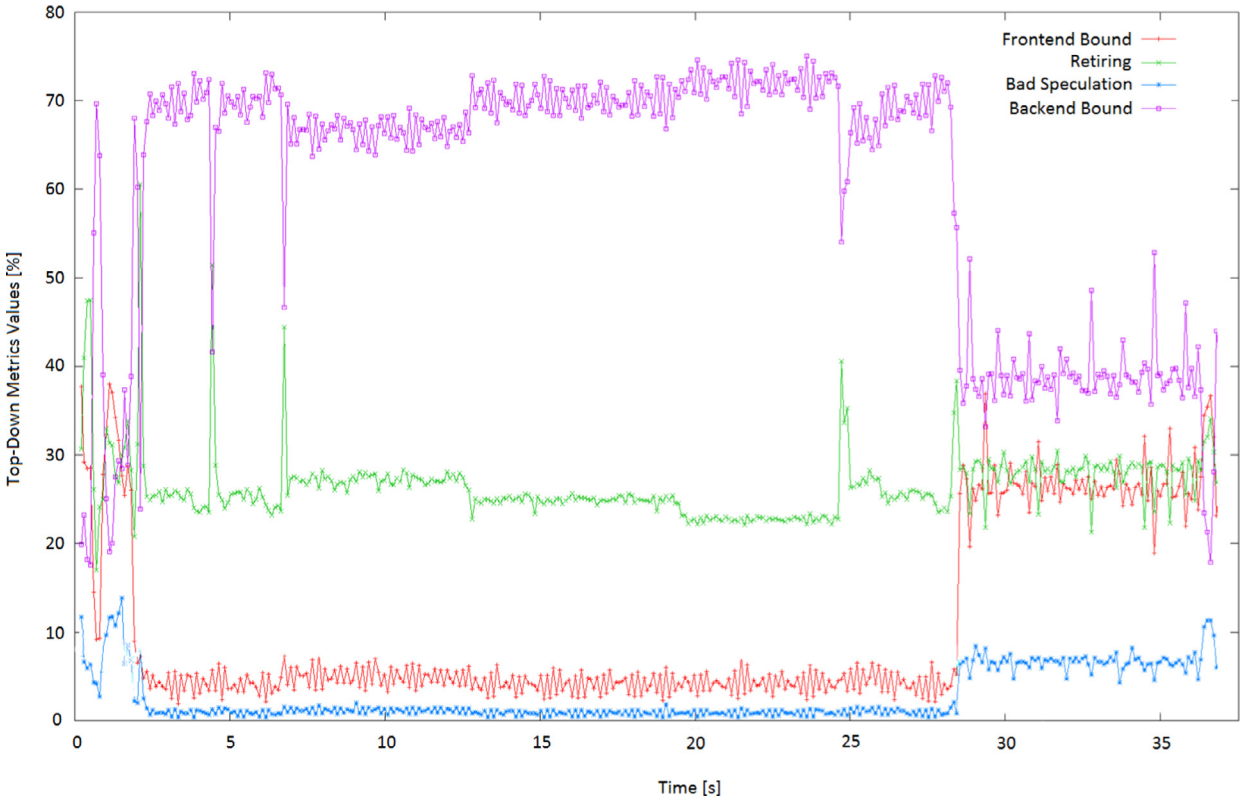


Fig. 5. Values of Retiring, Frontend Bound, Bad Speculation and Backend Bound metrics for the leslie3d benchmark.

Algorithm 1 First, memory inefficient implementation of the matrix multiplication (later called for1 in this paper).

```
1: for (int i = 0; i < ROWS; ++i) do
2:   for (int j = 0; j < COLUMNS; ++j) do
3:     for (int k = 0; k < COLUMNS; ++k) do
4:       matrix_r[i][j] = matrix_r[i][j] + matrix_a[i][k] * matrix_b[k][j];
5:     end for
6:   end for
7: end for
```

In the actual experiments the ROWS and COLUMNS variables had a value of 50,000, while in the **for** loops they were divided by 1000 to decrease the time of the tests.

In the second implementation only one change is introduced in the line where matrix_*b* is accessed. The fastest changing array index in matrix_*b* is the second one. This change is presented in Algorithm 2.

Algorithm 2 Description of the change in the way the matrix_*b* is accessed in the second implementation of the matrix multiplication (later called for2 in this paper).

```
1: ...
2: matrix_r[i][j] = matrix_r[i][j] + matrix_a[i][k] * matrix_b[j][k];
3: ...
```

The values of Retiring, Backend Bound, Frontend Bound and Bad Speculation metrics are presented in Fig. 6. Easily visible is very high value of Backend Bound metric – 91.79% in the first implementation and 67.66% in the second one. The decrease in the Backend Bound value was reflected by Retiring metric. The efficiency of execution increased from 7.52% in the first implementation to 30.12% in the second case. Bad Speculation increased slightly (from 0.31% to 1.11%), as well as Frontend Bound (from 0.38% to 1.11%).

Memory Bound and Core Bound metrics are presented in Fig. 8. They clearly indicate huge improvement in memory accesses. In the first case 24.49% of CPU time was spent on waiting for data to be retrieved from memory. In the second case it dropped to only 1.01%. Core Bound was similar in both cases – 67.31% and 66.65%.

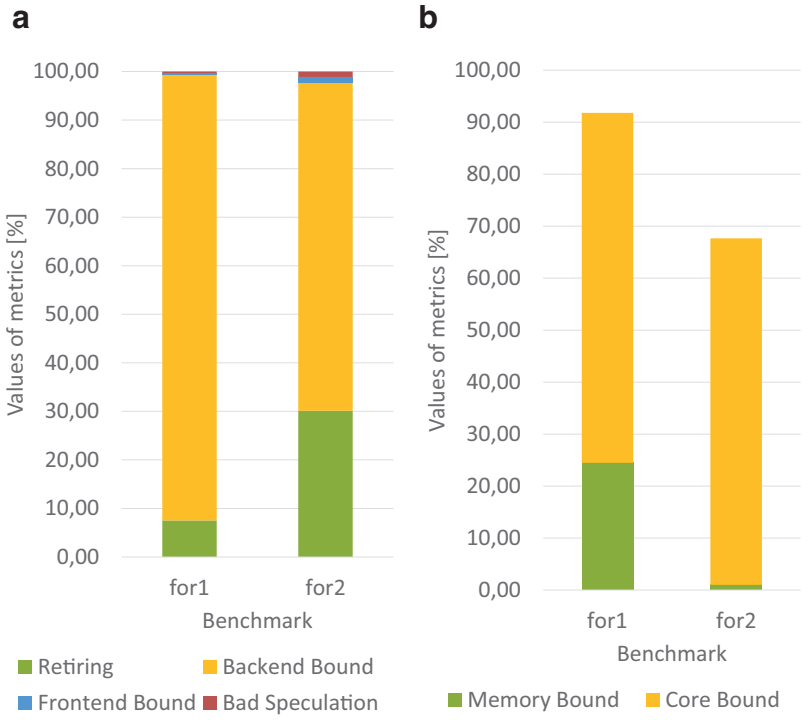


Fig. 6. Matrix multiplication: (a) Retiring, Backend Bound, Frontend Bound and Bad Speculation metrics, (b) Memory Bound and Core Bound metrics.

4.4. Analysis of multiple cores

The AMD Top-Down Characterization metrics may be used to analyze both single-threaded and multithreaded applications. Fig. 7 presents Retiring, Frontend Bound, Bad Speculation and Backend Bound metrics for four different executions of the pbzip2 benchmark, a parallel implementation of the bzip2 block-sorting file compressor. Fig. 8 presents Memory Bound and Core Bound results for the same four executions.

The first run is an execution of pbzip2 benchmark on one core. Three other cores of the CPU were turned off. Second run presents an execution of pbzip2 benchmark on each core of two available CPUs. Therefore, two cores were enabled, one for each CPU. The remaining two cores were turned off. The values of the metrics almost did not change: in both cases Frontend Bound had a value around 18%, Backend Bound – 49%, Retiring – 23.5% and Bad Speculation – 9.5%. These two cores worked independently, each servicing its own requests and the results of running a parallel benchmark on two different cores is similar to running one benchmark on one core. Fig. 8 shows that Memory Bound and Core Bound metrics have also the same values.

However, a noticeable change is visible in the third execution. It presents a case where *two* pbzip2 benchmarks are executed on one core. The Retiring and Bad Speculation metrics remained at the same levels (around 23.5% and 9.5%, respectively) but Frontend Bound dropped to 9%. It is mainly a result of a decrease in ICache Misses. The explanation of this behavior may be that if two identical threads are run side by side, the processor keeps in cache the decoded instructions from running the first thread which are later used by the second thread. This decreases the amount of ICache Misses. The reduction of Frontend Bound value affected Backend Bound – from 49% to 58%, keeping the processor busy for a longer time during execution. As seen in Fig. 8, Memory Bound also increased from 8% to 10%. Two threads on the same core compete for the same cache memories, thus increasing cache misses and causing more external memory requests.

Last, fourth execution presents a case where two threads of one pbzip2 benchmark are run on two cores of the same CPU. It differs from the second case where also two cores were used, but from two different CPUs. Since the instruction decoder is shared between cores, the Frontend efficiency increases even more in this case – Frontend Bound value decreased to around 6% (compared to almost 18% in the first and second case). The Retiring category was similar, though it dropped a little to the least value out of these four cases presented – 23.14%, compared to 23.66% in the “most efficient” case. This phrase was included in quotes, as the Retiring metric does not take into account the time of execution. It indicates only the efficiency of available resources. Although the run time decreased significantly, the overall efficiency understood as the percentage of retired microoperations in a unit of time dropped a little. Though the Frontend became more effective, the Backend had more work to do. This time the two threads were executed on two different cores which have separate caches, therefore the Memory Bound metric was similar as in the first and second case. Due to the increased number of microoperations passed to Backend, the Core Bound metric increased to 63%.

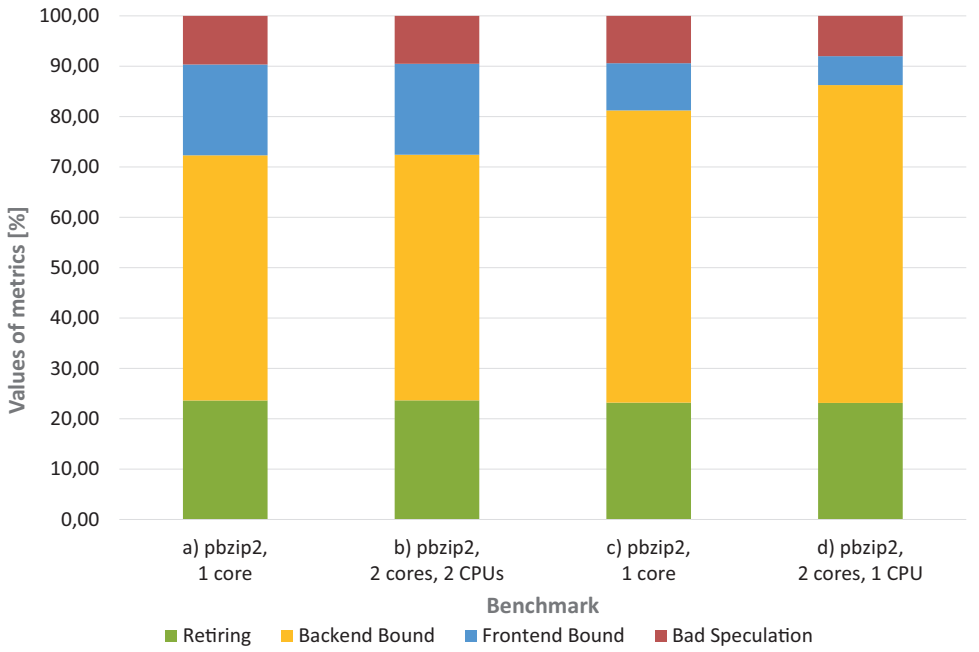


Fig. 7. Retiring, Backend Bound, Frontend Bound and Bad Speculation metrics for: (a) pbzip2 benchmark executed on one core, (b) pbzip2 benchmark executed on two cores of two different CPUs, (c) two pbzip2 benchmarks executed concurrently on one core, (d) pbzip2 benchmark executed on two cores of the same CPU.

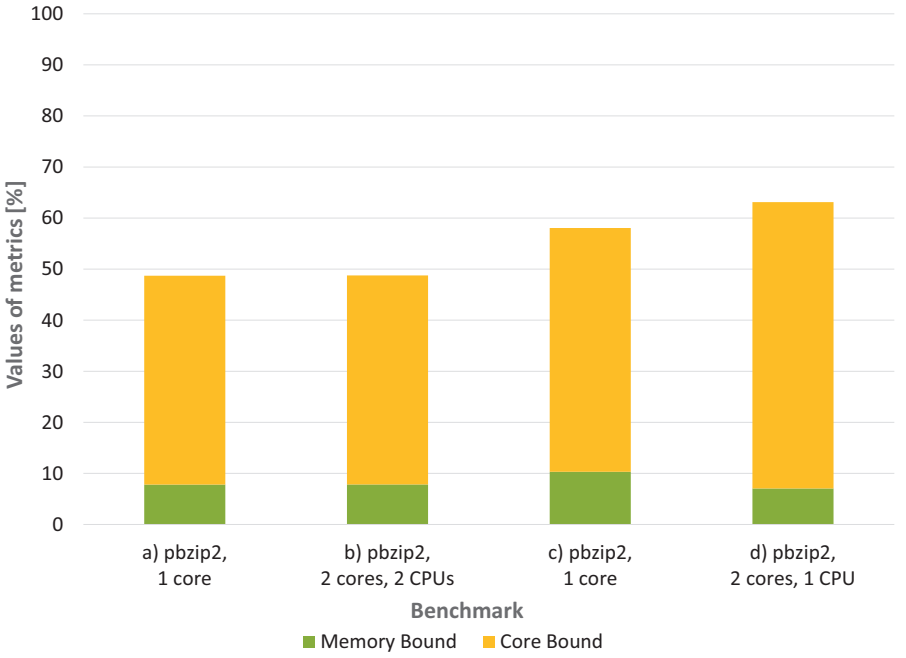


Fig. 8. Memory Bound and Core Bound metrics for: (a) pbzip2 benchmark executed on one core, (b) pbzip2 benchmark executed on two cores of two different CPUs, (c) two pbzip2 benchmarks executed concurrently on one core, (d) pbzip2 benchmark executed on two cores of the same CPU.

4.5. Microarchitectures comparison

Very interesting is a comparison of different architectures executing the same benchmarks. In this section we show a comparison between previously described processor AMD R and AMD Opteron 6234. Although these are processors designed for different markets (AMD R is an embedded processor, while AMD Opteron is a server processor), the AMD Top-Down Characterization may be used in both cases.

Table 4
AMD Opteron system parameters for the tests.

Processor	AMD Opteron 6234, 2.4GHz frequency, 12 cores, 6 x 64KB L1 instruction cache, 12 x 16KB L1 data cache, 6 x 2MB L2 data cache, 2 x 8MB L3 data cache
Memory	62GB DDR3 @ 1600MHz
OS	Linux Ubuntu 15.10
Benchmark	SPEC 2006CPU v1.2
Compiler	gcc v5.2.1

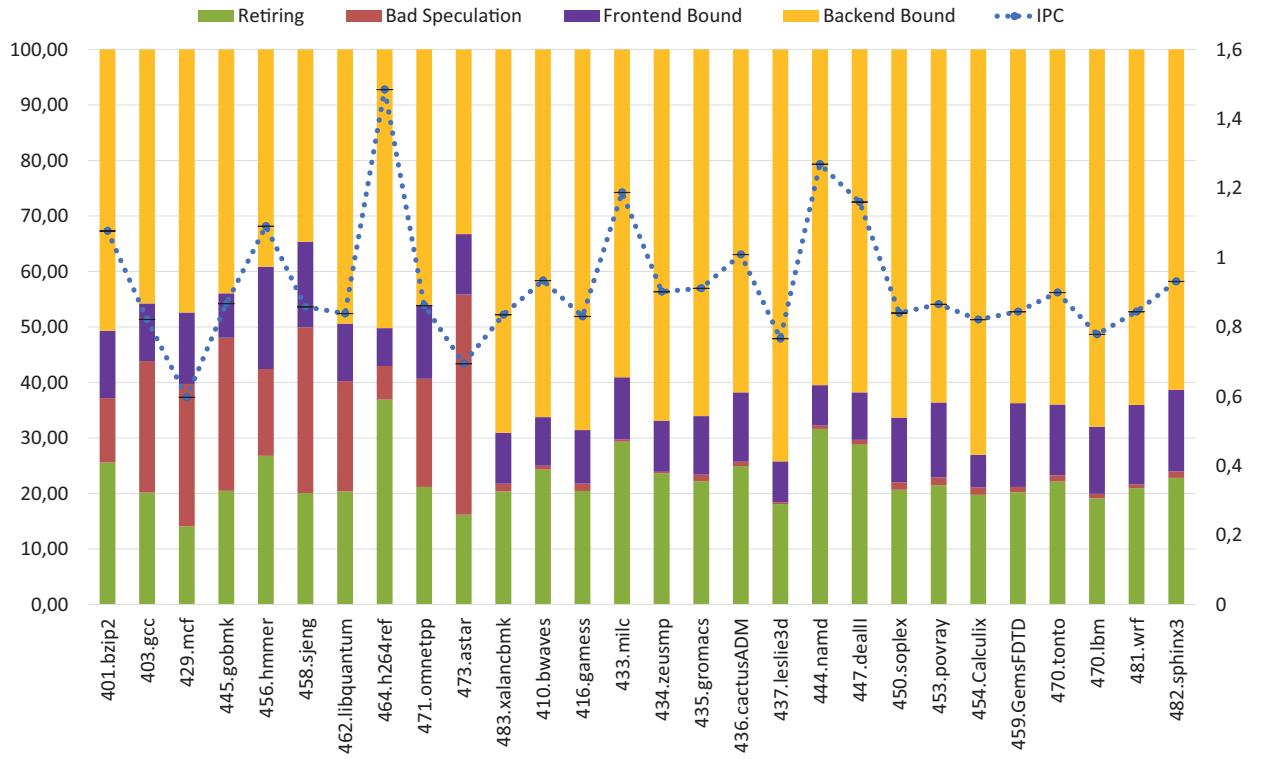


Fig. 9. Retiring, Backend Bound, Frontend Bound, Bad Speculation and IPC metrics for the SPEC CPU2006 suite on AMD Opteron processor.

The AMD Opteron 6234 specification is presented in Table 4. Compared to AMD R, this processor has additional 8MB shared data cache. Since Enternal Memory Bound metric takes into account L2 misses, cycles spent while waiting to retrieve data from L3 cache are also included in its value. Therefore, in case of AMD Opteron the Memory Bound metric takes into account also L3 cache accesses.

The values of the four top metrics for the SPEC CPU2006 suite for AMD Opteron processor are presented in Fig. 9.

For the analyzed benchmarks the Retiring metric has a value between 14.07% (mcf benchmark) and 36.89

Fig. 10 shows the Retiring, Backend Bound, Frontend Bound, Bad Speculation and IPC metrics for the SPEC CPU2006 suite on Intel processor from the publication introducing the Intel Top-Down method [1]. Easily visible is a similar distribution of the IPC metric across benchmarks (and thus the Retiring metric, as it correlates with IPC). For example, for the first three benchmarks it goes down, then it increases until hmmer and goes down for sjeng. For libquantum the values are very different for some reason – in case of AMD processor it is quite low, while for Intel it has the highest value. But the lowest values are similar in both cases. For Intel processor the IPC value is the lowest for omnetpp and mcf benchmark. For AMD mcf has the lowest value, while omnetpp is also among the benchmark with low IPC.

As noted in [1], the Integer applications are more sensitive to Frontend Bound and Bad Speculation than the FP applications for Intel processor. The same applies to AMD. In case of Bad Speculation the difference is easily visible – it is almost negligible for FP benchmarks. In both processors the highest value of Bad Speculation has astar.

Fig. 11 presents the results for Memory Bound and Core Bound metrics. Since the processor has additional L3 cache, presented benchmarks are less memory bounded than in the case of AMD R. Most of the computations are Core Bounded.

In this paragraph we will analyze in more detail the most memory-bounded benchmark – milc. Fig. 12a presents a comparison of Retiring, Frontend Bound, Bad Speculation and Backend Bound metrics for the milc benchmark only on both processors, while Fig. 12b presents a comparison of Memory Bound and Core Bound metrics for these two machines. The

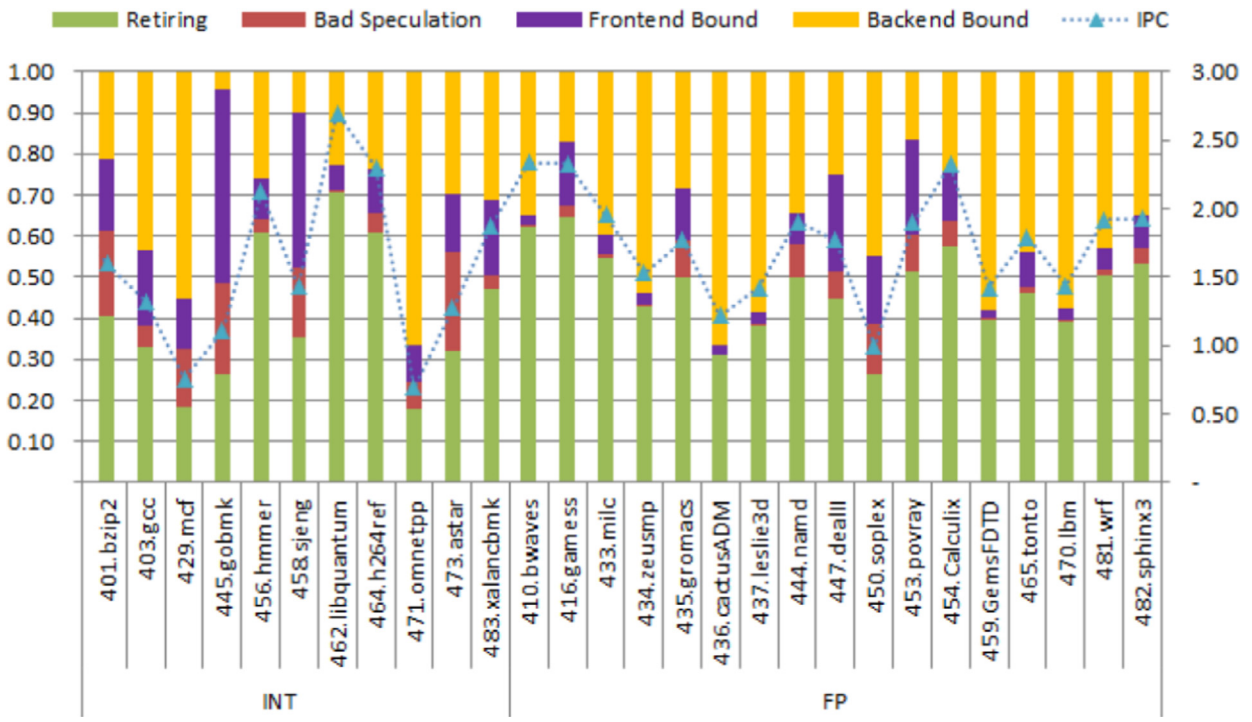


Fig. 10. Retiring, Backend Bound, Frontend Bound, Bad Speculation and IPC metrics for the SPEC CPU2006 suite on Intel processor [1].

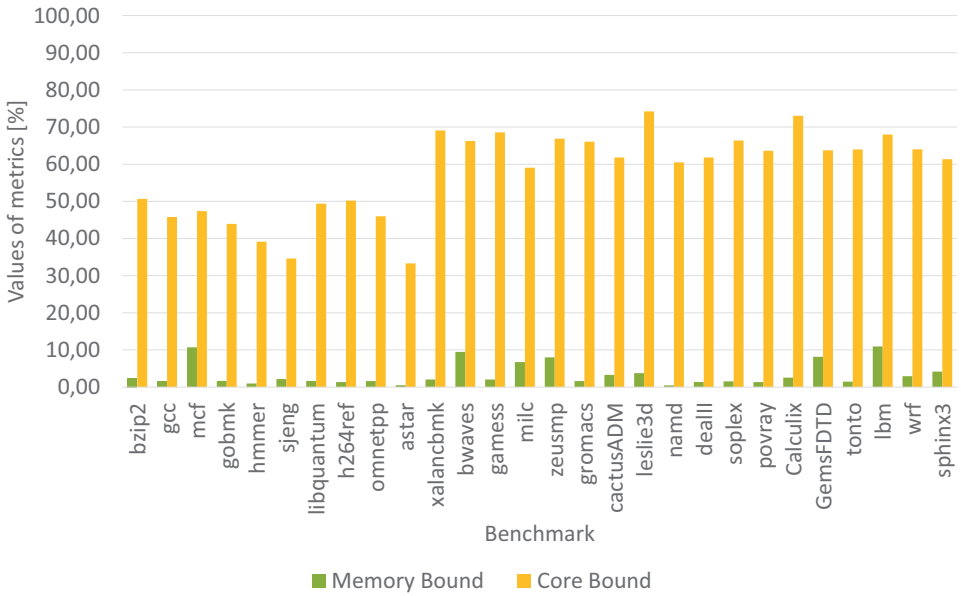


Fig. 11. Memory Bound and Core Bound metrics for the SPEC CPU2006 suite on AMD Opteron processor.

execution time decreased from 31 s on AMD R to 22 s on AMD Opteron and the Retiring metric increased by one fourth – from 23.42% on AMD R to 29.33% on Opteron. Bad Speculation metric is almost the same in both cases – around 5%. The Frontend of the processor is also more efficient, as the Frontend Bound metric has a value of 11.19%, compared to 17.28% in AMD R. Finally, AMD Opteron has almost the same Backend Bound metric – around 55%. Most of the calculations are therefore Backend Bounded. Taking a look at Fig. 12b shows an improvement in Memory Bound metric (its value is 6.72% for AMD Opteron, compared to 23.23% on AMD R). It is not surprising, since AMD Opteron has an additional L3 cache, which is much faster than the RAM memory. Most of the computations are Core Bounded.

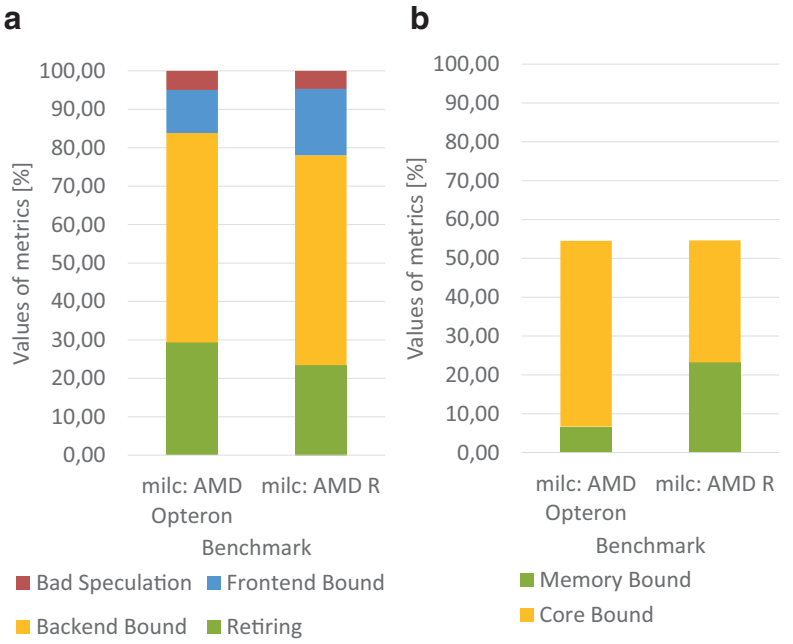


Fig. 12. Comparison of milc benchmark executed on AMD Opteron 6234 and AMD R 464L: (a) Retiring, Backend Bound, Frontend Bound and Bad Speculation metrics, (b) Memory Bound and Core Bound metrics.

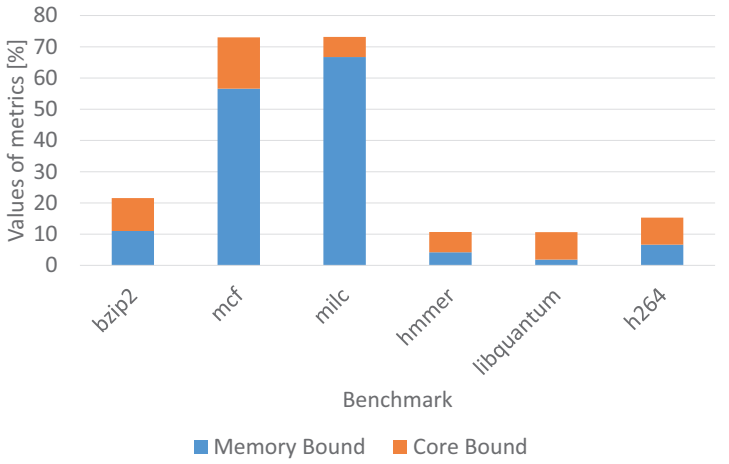


Fig. 13. Memory Bound and Core Bound metrics for a few selected benchmarks on Intel Core i7-4700EQ processor.

4.6. Energy consumption

Since the metrics are calculated by taking into account mainly the stall cycles of the processor, optimizing their values (in other words: reducing the boundedness of the application) leads indirectly to optimizing the energy usage of executed applications. For example, the execution time of the first implementation of matrix multiplication takes 23 s, while second implementation – only 5 s. Moreover, the optimized version of the matrix multiplication program takes 67% less energy than its non-optimized counterpart. Presented metrics do not directly indicate the energy efficiency improvement, though it is definitely an interesting future research topic.

In this section the Intel Core i7-4700EQ processor was selected for the tests, instead of AMD CPU. It is due to the fact that this processor allows to retrieve information from hardware about the energy used by DRAM and processor core separately, in contrast to the AMD processors analyzed in this paper. However, as the future AMD Zen processors will also allow to retrieve such information, therefore suggested experiments may be repeated on AMD CPUs in the future as well.

Fig. 13 presents the Memory Bound and Core Bound metrics for a few selected benchmarks on Intel Core i7-4700EQ processor. On the other hand, Fig. 14 presents the energy used by DRAM and core in the same experiments. Since this figure

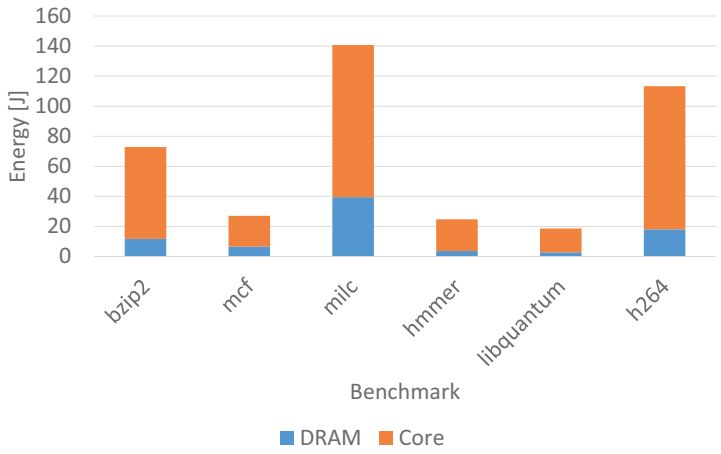


Fig. 14. Energy usage of memory and core for a few selected benchmarks on Intel Core i7-4700EQ processor.

presents the whole energy used, benchmarks having similar values of metrics (e.g. bzip2 and h264) may have definitely different values of energy consumed – metrics show the percentage of time a given part of processor was inefficient, while energy also contains the information about the duration of a given program execution. However, if only the portion of Memory Bound metric in the whole Backend Bound value was compared to the portion of DRAM energy to the whole energy used by DRAM and core, some correlations are visible. For example, the Memory Bound metric divided by Backend Bound metric for h264 and bzip2 gives 0,43 and 0,51, respectively. Energy of memory divided by the sum of energy of memory and core for h264 and bzip2 gives 0.159 J and 0.161 J, respectively. On the other hand, milc benchmark is the most memory bounded one in this set. Memory Bound divided by Backend Bound gives 0,91 (around 45–50% higher than in the previous two benchmarks). This high value is also represented by high value of energy usage in the whole energy consumption – energy of memory divided by the whole energy used by memory and core is 0.29 J (also around 45% more than in the case of two previous benchmarks).

More detailed analysis of correlation between energy consumption and the values of metrics is out of scope of this paper. This section, however, shows that some correlations exist and it is possible to further analyze this research question.

5. Related work

This work is based on the Intel Top-Down Method originally presented in [1]. To our knowledge, it is the first attempt to transfer this method to AMD processors.

The original Intel method has more metrics and gives more insight into the internals of the processor due to the availability of a larger number of performance counters. However, presented characterization may be expanded in the future if the AMD processors will be enriched with new performance counters.

In the past there were multiple approaches to performance evaluation. The most known were naïve approaches, such as [8,9]. While they gave correct results for in-order CPUs, the out-of-order processors introduce more complex mechanisms during execution, such as overlapping stalls, speculative executions or different penalties for different workloads, making the previous approaches imprecise.

In some cases, such as [10,11], some algorithms are used or simulations are performed to predict the exact performance of designs. Still, they require to build a model by running multiple, usually slow simulations to explore large processor design space. They usually also do not give detailed insight into the internals of a specific execution.

Some authors, such as [12,13], propose methods to create *Cycles per Instruction stacks* for superscalar processors. They break down a single-thread programs execution time into a number of cycle components which represent how many cycles the program is stalled due to various miss events, such as cache and TLB misses. In referenced papers the authors utilize interval analysis. The basis for the model is the observation that under *ideal* conditions, a superscalar processor can sustain an instruction per cycle (IPC) rate that is roughly equal to the superscalar pipeline width. Still, presented approaches are based on analytical modeling and do not give insight into the details of a given architecture.

Other profiling mechanisms have been proposed for out-of-order processors. In [16] the authors propose the *ProfileMe* framework which is based on per-instruction profiling. It creates a detailed record of all interesting events and pipeline stage latencies as a sampled instruction moves through the processor pipeline. The drawback of this approach is that it does not allow to model overlap effects. This limitation was later addressed by Fields et al. [17] by proposing the shotgun profiling. It is a class of hardware profiling infrastructures that are parallelism-aware. Still, the per-instruction profiling relies on sampling which may introduce inaccuracy. The overlap effects are computed in a postmortem analysis. The amount of overlaps and interactions between instructions are determined based on a simple processor model.

Some researchers have attempted to correctly define the scalability bottlenecks by defining *speedup stacks*. They report how much each scalability bottleneck limits the scalability of a multi-threaded program. Eyerman et al. [14] achieve high accuracy in their method, though their method requires either simulating real multicore hardware or requires additional hardware which is not available in products of major chip manufacturers. In [15] the authors propose a method for creating speedup stacks that does not require simulations and uses existing performance counters. In case of multithreaded applications, it requires additional framework which overloads the Pthread functions to take control of the executed threads. Their implementation does not handle condition variables and may be used only with applications that are explicitly parallelized with Pthreads.

Multiple researchers created models of a superscalar processor. In [18] the authors created a model that models the relationship between instructions issued per cycle and the size of the instruction window under ideal conditions. They also propose methods for calculating transient performance penalties due to branch mispredictions, instruction cache misses, and data cache misses. Chen and Aamodit [19] improved the first-order superscalar processor model by considering the cost of pending hits, data prefetching and MSHRs. They proved that modeling prefetching and MSHRs significantly increase the accuracy of the first-order processor model. However, they only showed memory instructions CPI results comparing with the results of a cycle accurate simulator.

6. Summary and future work

This paper presented AMD Top-Down Characterization Approximation which is a methodology to identify true bottlenecks in out-of-order processors. As not all of the performance counters are available on AMD processor to calculate the exact values of metric, the methodology was named as an approximation. Due to the missing performance counters it is impossible to calculate the exact error. Described metrics correspond to the efficiency of different parts of the processor and are calculated using the values of performance counters. The method may be used to analyze the execution of both single and multithreaded applications, across a variety of workloads. This method may be used in a variety of situations, from optimizing the applications during development phase, to detailed analysis of their execution, to detailed investigation of processor efficiency.

The authors hope that this method will be an incentive for AMD to enrich their future processors with more performance counters that will further enrich this method to give more insight into the internals of execution. Presented categories may be in the future further divided into subcategories that would give even more detailed information.

Moreover, the authors would like to further investigate the energy gain of optimization based on presented metrics to create a detailed energy profile of executed benchmarks.

Acknowledgment

The authors would like to thank Piotr Brona and Adrian Stelmaszyk for their help in configuring the servers and the anonymous reviewers for their valuable comments and suggestions.

The research presented in this paper is funded by a Grant from Polish National Science Center under award number 2013/08/A/ST6/00296.

References

[1] A. Yasin, A top-down method for performance analysis and counters architecture, in: Proceedings of the 2014 IEEE International Symposium on Performance Analysis of Systems and Software, IEEE, 2014, pp. 35–44.
[2] A. Fog, The microarchitecture of Intel, AMD and via cpus. an optimization guide for assembly programmers and compiler makers, 2016. [Online; posted 16 January 2016] URL http://www.agner.org/optimize/microarchitecture.pdf.
[3] S. Eyerman, L. Eeckhout, A counter architecture for online dvfs profitability estimation, IEEE Trans. Comput. 59 (2010) 1576–1583.
[4] G. Keramidas, V. Spiliopoulos, S. Kaxiras, Interval-based models for run-time DVFS orchestration in superscalar processors, in: Proceedings of the Seventh International Conference on Computing Frontiers, ACM, 2010, pp. 287–296.
[5] B. Rountree, D. Lowenthal, M. Schulz, B.R. de Supinski, Practical performance prediction under dynamic voltage frequency scaling, in: Proceedings of the International Green Computing Conference and Workshops (IGCC), IEEE, 2011, pp. 1–8.
[6] B. Su, J.L. Greathouse, J. Gu, M. Boyer, L. Shen, Z. Wang, Implementing a leading loads performance predictor on commodity processors, in: Proceedings of the USENIX Annual Technical Conference, USENIX ATC 14: 2014, 2014, pp. 205–210.
[7] P.J. Drongowski, Instruction-based sampling: a new performance analysis technique for AMD family 10h processors, 2007, [Online; posted 16 November 2007] URL http://developer.amd.com/wordpress/media/2012/10/AMD_IBS_paper_EN.pdf.
[8] A. Ailamaki, D. DeWitt, M.D. Hill, D.A. Wood, DBMSs on a modern processor: Where does time go? in: Proceedings of the Twenty Fifth International Conference on Very Large Data Bases, VLDB '99, 1999, pp. 266–277.
[9] D. Levinthal, Performance analysis guide for Intel core i7 processor and Intel Xeon 5500 processors, 2009. [Online] URL https://software.intel.com/sites/products/collateral/hpc/vtune/performance_analysis_guide.pdf.
[10] J. Lee, H. Jang, J. Kim, Rpstacks: fast and accurate processor design space exploration using representative stall-event stacks, in: Proceedings of the Forty Seventh Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-47, IEEE Computer Society, Washington, DC, USA, 2014, pp. 255–267.
[11] D. Shen, O. Luo, G.M. Poshyvanyk, Automating performance bottleneck detection using search-based application profiling, in: Proceedings of the 2015 International Symposium on Software Testing and Analysis, ISSTA 2015, ACM, 2015, pp. 270–281.
[12] S. Eyerman, L. Eeckhout, S.J. Karkhanis, A performance counter architecture for computing accurate CPI components, in: Proceedings of the Twelfth International Conference on Architectural Support for Programming Languages and Operating Systems ASPLOS XII, ACM, 2006, pp. 175–184.
[13] T. Karkhanis, J. Smith, Automated design of application specific superscalar processors: an analytical approach, in: Proceedings of the Thirty Fourth Annual International Symposium on Computer Architecture, ISCA '07, ACM, 2007, pp. 402–411.
[14] S. Eyerman, K.D. Bois, L. Eeckhout, Speedup stacks: Identifying scaling bottlenecks in multi-threaded applications, in: Proceedings of the 2012 IEEE International Symposium on Performance Analysis of Systems and Software, IEEE, 2012, pp. 145–155.

[15] D. Eklov, N. Nikoleris, E. Hagersten, A Profiling Method for Analyzing Scalability Bottlenecks on Multicores, Technical Report, Department of Information Technology, Uppsala University, 2012.
[16] J. Dean, J.E. Hicks, C.A. Waldspurger, W.E. Weihl, G. Chrysos, Profileme: hardware support for instruction-level profiling on out-of-order processors, in: Proceedings of the Thirteenth Annual IEEE/ACM International Symposium on Microarchitecture, Research Triangle Park, NC, 1997.
[17] B.A. Fields, R. Bodik, M.D. Hill, C.J. Newburn, Interaction cost and shotgun profiling, ACM Trans. Archit. Code Optim. 1 (3) (2004) 272–304.
[18] T.S. Karkhanis, J.E. Smith, A first-order superscalar processor model, Proceedings of the Thirty First Annual International Symposium on Computer Architecture, 32(2), 2004, p. 338.
[19] X.E. Chen, T.M. Aamodt, A first-order superscalar processor model, in: Proceedings of the Fifteenth International Symposium on High Performance Computer Architecture, 2009, pp. 329–340.